

TransitFlow 正規化與資料庫設計說明

1. 正規化總覽

正規化 (normalisation) 的目標是降低資料重複、避免更新異常 (update anomaly)、插入異常 (insert anomaly) 與刪除異常 (delete anomaly)。TransitFlow 的核心交易資料採用正規化設計，讓使用者、車站、班次、停靠站、訂票與付款分別存放在清楚的 entity 中。

Normal Form	TransitFlow 的應用
1NF	欄位盡量維持單一值；交易核心欄位如 amount_usd、status、travel_date、departure_time 都是 atomic value。
2NF	複合主鍵表如 schedule_stops 使用 (schedule_id, stop_order)，非鍵欄位 station_id 與 travel_time_from_origin_min 依賴整個複合鍵，而不是只依賴其中一部分。
3NF	bookings 不重複儲存 user_email、station_name、schedule line 等可由 FK 查得的資料，避免非鍵欄位依賴另一個非鍵欄位。

2. 3NF 設計決策一：Schedule Stops 使用 Junction Table

TransitFlow 沒有把一個班次的所有停靠站存成單一 array 欄位，而是建立 national_rail_schedule_stops 與 metro_schedule_stops。這是最重要的正規化決策之一，因為班次與車站是多對多關係，而且站序本身也是關係屬性。

資料表	主鍵 / Candidate Key	Functional Dependency
national_rail_schedule_stops	PK = (schedule_id, stop_order); UNIQUE = (schedule_id, station_id)	(schedule_id, stop_order) → station_id, travel_time_from_origin_min
metro_schedule_stops	PK = (schedule_id, stop_order); UNIQUE = (schedule_id, station_id)	(schedule_id, stop_order) → station_id, travel_time_from_origin_min
national_rail_schedules	schedule_id; schedule_code 為 unique code	schedule_id → line, service_type, direction, origin_station_id, destination_station_id, operates_on
metro_schedules	schedule_id; schedule_code 為 unique code	schedule_id → line, direction, origin_station_id, destination_station_id, base_fare_usd, per_stop_rate_usd

如果把 stops 存在 schedules.stops_in_order array，資料庫無法直接用 FK 保證每個 stop 都存在於 stations，也較難檢查 stop_order 是否重複。拆成 junction table 後，每一列只描述「某個 schedule 的第幾站是哪個 station」，因此關係更清楚，也能讓 SQL 查詢直接用 JOIN、ORDER BY stop_order 取得完整路線。

這個設計達成 3NF：station_name 沒有放在 schedule_stops，因為 station_id → station_name，而 station_name 應該存在 stations 表。如果 schedule_stops 同時存 station_id 與 station_name，就會出現 transitive dependency：(schedule_id, stop_order) → station_id → station_name，違反 3NF。

3. 3NF 設計決策二：Bookings 不重複儲存使用者與車站資訊

national_rail_bookings 與 metro_trips 是交易紀錄，但它們不直接存 user email、user name、origin station name 或 destination station name，而是使用 FK 連到 registered_users、stations 與 schedules。

不建議的欄位放法	問題	目前 schema 的做法
booking_id, user_email, full_name, station_name, amount_usd	user_email → full_name; station_id → station_name, 這些不是由 booking_id 直接決定，會造成 transitive dependency。	bookings 只存 user_id、origin_station_id、destination_station_id，需要顯示名稱時再 JOIN。
metro_trips 直接存 schedule line 與 direction	schedule_id → line, direction; 若直接存會在班次資料更新時造成重複更新。	metro_trips 只存 schedule_id。
payments 直接重複存 user name 與 ticket details	付款只需要連到父交易；重複交易細節會增加不一致風險。	payment table 透過 booking_id / trip_id / pass_id 連回父交易。

以國鐵訂票為例，booking_id 是主鍵，合理的 functional dependency 是：booking_id → user_id, schedule_id, origin_station_id, destination_station_id, travel_date, departure_time, seat_pk, amount_usd, status。使用者名稱與 email 則是 user_id → full_name, email；車站名稱則是 station_id → name。因此 booking 表不應直接保存 full_name 或 station name。

4. 3NF 設計決策三：付款與回饋依交易類型拆表

TransitFlow 將付款表拆成 national_rail_payments、metro_payments 與 metro_monthly_pass_payments，也將 feedback 拆成 national_rail_feedback 與 metro_feedback。這個設計比單一 payments(transaction_ref, transaction_type) 更符合關聯式資料庫的完整性要求。

資料表	父交易 FK	設計理由
national_rail_payments	booking_id → national_rail_bookings.booking_id	付款一定屬於一筆國鐵訂票，FK 可由資料庫強制檢查。
metro_payments	trip_id → metro_trips.trip_id	捷運單程 / 日票旅程付款與國鐵訂票分開，避免 transaction_ref 指錯表。
metro_monthly_pass_payments	pass_id → metro_monthly_passes.pass_id	月票是獨立購買紀錄，不應混入 metro_trips。
national_rail_feedback	booking_id → national_rail_bookings.booking_id	國鐵回饋必須對應國鐵旅程。
metro_feedback	trip_id → metro_trips.trip_id	捷運回饋必須對應捷運旅程。

如果使用單一 transaction_ref 欄位，資料庫無法建立「這個 ref 必須存在於 bookings 或 metro_trips 或 monthly_passes 其中之一」的標準 FK。拆表後，每張 payment table 都有明確父表，能避免 orphan payment，也能讓 ERD cardinality 更清楚。

5. 刻意反正規化與取捨

正規化不是永遠越多越好。TransitFlow 有幾個地方刻意保留較簡化或半結構化的資料，以換取查詢簡單度與效能。重點是這些反正規化不會破壞核心交易資料的一致性。

設計選擇	可能的完全正規化做法	目前選擇與理由
national_rail_seats 直接存 coach、fare_class、seat_row、seat_column	拆成 layout、coaches、seats 三張表。	座位資料主要用於查詢可用座位，扁平化後可直接透過 schedule_id、fare_class 查詢，不必多次 JOIN。由於座位配置在 seed 後通常不常變更，重複 coach / fare_class 的成本可接受。
stations.lines 使用 JSONB	建立 station_lines junction table。	lines 是車站主資料中的半結構屬性，主要用於顯示與少量查詢；使用 JSONB 可保留原始資料結構並搭配 GIN index。
adjacent_stations 使用 JSONB	建立 station_adjacencies 表。	關聯式 ERD 已表達交易與主資料；路網最短路徑實際交給 Neo4j 處理，所以 PostgreSQL 中保留 JSONB 作為匯入與展示資料。
amount_usd 存在 booking / trip / payment	每次由 fare table 即時計算。	付款金額屬於 transaction snapshot。票價未來可能調整，歷史付款紀錄必須保留當時實際金額。

其中 national_rail_seats 是最明顯的 de-normalisation trade-off。理論上 coach 的 fare_class 可能可以抽到 coach table，但本專案的主要查詢是「某日期、某班次、某票種還有哪些座位可訂」。扁平化設計讓 get_available_national_rail_seats() 可以從 national_rail_seats 排除已被 booking 佔用的 seat_pk，查詢更直接。

6. Password Hashing 設計：Argon2id

registered_users 不儲存明文 password，而是儲存 password_hash。schema 中 password_hash 欄位有 CHECK (password_hash LIKE \$argon2id\$%)，表示實際儲存值必須是 Argon2id 格式的雜湊字串。

Argon2id 是現代密碼雜湊演算法，適合用於密碼儲存，原因不是單純「安全」而已，而是它具備 memory-hard 與 key stretching 特性。Memory-hard 代表攻擊者大量用 GPU 平行暴力破解時會受到記憶體成本限制；key stretching 則讓每次猜密碼都需要花費可控的計算時間。

比較項目	MD5 / SHA-1	bcrypt	Argon2id
設計用途	一般雜湊 / 完整性檢查，不適合存密碼。	密碼雜湊，使用 cost factor 增加計算成本。	密碼雜湊競賽 winner，結合時間成本與記憶體成本。
速度	太快，攻擊者可大量暴力破解。	可調慢，但主要是 CPU cost。	可調 time_cost、memory_cost、parallelism，更能抵抗 GPU。
Salt	若自行使用常常容易漏掉或做錯。	雜湊字串內包含 salt。	雜湊字串內包含 salt 與參數。
本專案選擇	不用，因為 plain hash 也不合格。	可接受，但本專案採用 Argon2id。	使用 password_hash 儲存完整 Argon2id encoded hash。

Salt 的作用是讓兩位使用者即使使用相同密碼，也會得到不同的 password_hash。Argon2id 的 encoded hash 會包含演算法名稱、參數、salt 與 hash 結果，因此資料庫不需要額外開 salt 欄位。這可以防止 rainbow-table attack，因為攻擊者不能用同一份預先計算表直接比對所有使用者。

登入時系統不會解密 password_hash, 因為雜湊不是加密, 不能還原。正確流程是把使用者輸入的密碼交給 Argon2 驗證函式, 讓函式讀取儲存在 hash 字串內的 salt 與參數, 重新計算並比對結果。

補充: 目前 schema 依照專案要求只有 password_hash 使用 Argon2id; secret_answer 仍是 VARCHAR。若此系統要進入 production, secret_answer 也應該雜湊, 或改用 email reset token / TOTP, 而不是保存可被猜測的秘密答案。

7. 資料庫術語對照

術語	中文說明	TransitFlow 例子
Functional dependency	某欄位或欄位組可以決定其他欄位。	booking_id → user_id, schedule_id, amount_usd, status。
Determinant	在 functional dependency 左側、負責決定其他欄位的欄位。	在 schedule_id → line, direction 中, schedule_id 是 determinant。
Candidate key	可以唯一識別 row 的候選鍵。	registered_users.email、registered_users.user_code 都是 unique, 可視為 candidate keys。
Primary key	實際選定用來唯一識別 row 的主鍵。	registered_users.user_id UUID、national_rail_schedules.schedule_id SERIAL。
Foreign key	指向另一張表 PK 或 unique 欄位的欄位。	national_rail_bookings.user_id → registered_users.user_id。
Partial dependency	非鍵欄位只依賴複合鍵的一部分, 違反 2NF。	若 schedule_stops 中 line 只依賴 schedule_id, 而不是 (schedule_id, stop_order), 就會是 partial dependency。
Transitive dependency	非鍵欄位依賴另一個非鍵欄位, 違反 3NF。	booking_id → station_id → station_name; 所以 booking 表不直接存 station_name。
Junction table	用來表示多對多關係的中介表。	national_rail_schedule_stops 連接 schedules 與 stations, 並保存 stop_order。
Composite key	由多個欄位組成的主鍵。	metro_schedule_stops 的 PK = (schedule_id, stop_order)。
De-normalisation	為效能或簡化查詢而保留部分重複或半結構資料。	national_rail_seats 扁平化 seat layout; stations.lines 使用 JSONB。

8. 總結

TransitFlow 的關聯式 schema 在核心交易部分採用明確的正規化設計: 使用者、車站、班次、停靠站、訂票、旅程、付款與回饋各自成為獨立 entity, 並透過 FK 維持資料完整性。Schedule stops 的 junction table 是最明確的 3NF 決策, 因為它移除了 stop array 造成的查詢與約束問題, 也避免 station_name 等傳遞相依資料被重複存放。

同時, 系統也在座位與 JSONB 屬性上做了有限度的反正規化, 以提升可用座位查詢與資料匯入展示的簡潔性。這些不影響付款、訂票、使用者資料的一致性。Password hashing 則使用 Argon2id, 透過 salt、memory-hard 與 key stretching 防止明文密碼外洩、rainbow table 與高速暴力破解。